

WeatherService.GetAsync()

Follow one request from the **Task** it promises, down the **try** happy path, and out through **catch** when something breaks. Same envelope comes back either way.

Order placed = the HTTP GET

Two doors = Data or Error, one type

Holding ≠ becoming = Data holds a WeatherInfo

01

Reading the signature

Before the body runs, the return type already tells the caller the deal.

```
public async Task<CardResult<WeatherInfo>> GetAsync()
```

async

This method **awaits** inside. It can hand control back at each I/O step and resume when that step finishes.

Task<...>

A **promise of a future value**. The method returns a Task *immediately*; the real CardResult lands inside it when the work completes.

CardResult<WeatherInfo>

The **envelope** that comes back. It carries **either** Data (a WeatherInfo) **or** Error (a string).

Who calls it? IndexModel does — it writes `Weather = await _weather.GetAsync();`. The `await` unwraps the Task and hands IndexModel a finished `CardResult<WeatherInfo>`.

HOLDING ≠ BECOMING The CardResult **holds** a WeatherInfo inside Data — it is **not** a WeatherInfo itself. To get the temperature you reach *through* the envelope: `result.Data.Summary`.

```
return CardResult<WeatherInfo>
```

ONE RETURN TYPE · TWO WAYS TO FILL IT

DOOR 1 · SUCCESS

```
{ Data = new WeatherInfo { ... } }
```

DOOR 2 · FAILURE

```
{ Error = "Couldn't load..." }
```

Error stays null. The card shows the forecast.

Data stays null. The card shows the error.

02

Inside `try` — the success path

Each step produces a concrete **type** that feeds the next. Blue notes mark where `await` waits on the network or disk; red tags mark lines that can throw.

● `await` = wait on this one op ● can throw → jumps to catch

1

```
var url = "https://api.open-meteo.com/..." → string
```

Write the order. Just the address plus query params — latitude, longitude, and which current fields you want (temperature, wind). Nothing has left the building yet.

2

```
await _httpClient.GetAsync(url) → HttpResponseMessage can throw
```

Place the order. The GET goes out over the network and comes back as an `HttpResponseMessage` (status + headers + body). `using` disposes it at block end.

`await` = wait for *this reply* to travel back. It does **not** freeze the app — the thread is released to serve other work, then resumes here when bytes arrive.

3

```
response.EnsureSuccessStatusCode() → void can throw
```

Was the order accepted? No `await`, no network — it just inspects the status. On 2xx it returns quietly and we continue. On 4xx/5xx it throws `HttpRequestException`, so a bad status can't slip through as "success".

4

```
await ...ReadAsStreamAsync() → Stream
```

Open the body as bytes. The response body becomes a `Stream` you can read from. `await using` disposes it asynchronously at block end.

`await` = wait until the body is ready to read. Again, waiting on *this* op, not blocking everything.

5

```
await JsonDocument.ParseAsync(stream) → JsonDocument can throw
```

Parse into a tree. The stream's bytes are read and parsed into a `JsonDocument` — an in-memory JSON tree you can navigate. `using` disposes it. Malformed JSON throws `JsonException`.

await = wait while the stream is drained and parsed. This is the last I/O wait — everything after is pure in-memory work.

6

```
doc.RootElement.GetProperty("current")
```

→ JsonElement

can throw

Navigate the tree. RootElement is the top JsonElement (the whole object).
.GetProperty("current") steps down into the current node — another JsonElement.
Missing key → KeyNotFoundException.

7

```
current.GetProperty("temperature_2m").GetDouble()
```

→ double

can throw

Pull the numbers. Same move for temperature and wind: get the property's JsonElement, then .GetDouble() converts it to a real double. If the node isn't a number, .GetDouble() throws InvalidOperationException.

8

```
new WeatherInfo { TemperatureC = temp, WindKph = wind, Summary = ... }
```

→ WeatherInfo

Fill the model. The two doubles become a plain WeatherInfo, and the interpolated string builds Summary (e.g. "21.3°C · wind 8.4 km/h"). This is the payload — not yet the envelope.

9

```
return new CardResult<WeatherInfo> { Data = ... }
```

→ CardResult<WeatherInfo> · Door 1

Seal the envelope and hand it back. The WeatherInfo goes into Data; Error stays null. This CardResult flows out through the Task to IndexModel. **Holding ≠ becoming:** the result *holds* the WeatherInfo — it isn't one.

03

What can throw — and why it lands in catch

Any exception anywhere in try **short-circuits the rest** — the remaining lines don't run — and control jumps straight to catch (Exception ex).

STEP	EXCEPTION	CAUSE
GetAsync	HttpRequestException	DNS / network down / timeout
EnsureSuccessStatusCode	HttpRequestException	API returned 4xx / 5xx

JsonDocument.ParseAsync	JsonException	body wasn't valid JSON
GetProperty("...")	KeyNotFoundException	expected field missing (shape changed)
GetDouble()	InvalidOperationException	value wasn't a number

04

Inside catch — Door 2, and why the page still loads

```
catch (Exception ex)
{
    return new CardResult<WeatherInfo>
    {
        Error = $"Couldn't load weather: {ex.Message}"
    };
}
```

It builds a `CardResult<WeatherInfo>` with `Error` set and `Data` null — the **same return type** as the success path. Because the exception is caught *here*, it never escapes to `IndexModel.GetAsync` **always** hands back a `CardResult`, never a thrown exception.

THE DEVPULSE PAGE
RENDERS ANYWAY

GitHub

Weather

Couldn't load
weather: ...

Crypto

Failure is isolated to one card. The other cards each own their own `CardResult`, so a weather outage shows a message in the weather card only — the page still loads.

05

How Index & Razor read the result

`IndexModel` awaits the `Task` to unwrap the `CardResult`; the view picks a door.

INDEXMODEL · UNWRAP THE TASK

```
Weather = await _weather.GetAsync();
// Weather is now a CardResult<WeatherInfo>
```

RAZOR · PICK A DOOR

```
@if (Model.Weather.Ok)
{
    <p>@Model.Weather.Data.Summary</p>
}
else
{
    <p>@Model.Weather.Error</p>
}
```

Ok is the convenience flag on `CardResult` — true when `Data` is present / `Error` is null. Success path → reach **through** the envelope for `Data.Summary` (holding ≠ becoming); failure path → show the plain `Error` string. One check decides which door's content renders.

```
// CHEAT SHEET - GLANCE WHILE YOU CODE HACKER NEWS
```

- 1 `async Task<CardResult<T>>` → caller `awaits` it; always gets a `CardResult`, never a thrown exception.
- 2 `try` = happy path: `GET` → `EnsureSuccess` → `ReadAsStream` → `JsonDocument` → `GetProperty` → `GetDouble` → `WeatherInfo` → `CardResult{ Data }`.
- 3 `await` = "wait for THIS I/O, free the thread" (reply, stream, parse) — not freeze the app.
- 4 Any throw in `try` → skip the rest → `catch` → `CardResult{ Error }`. Same type; page survives, only the card shows it.
- 5 Razor: `Ok ? Data.Summary : Error`. `CardResult` *holds* `WeatherInfo` in `Data` — it isn't one; reach through `.Data`.